

Reto de Latencia Mínima — Documentación técnica e informe de resultados

Diplomado en Arquitectura de Software y Cloud Computing · Pontificia Universidad Javeriana Cali

Módulo 1 — Fundamentos de la Arquitectura de Software · Group 2

Daniel Mazo Serna · Freddy Aparicio Marín · Bryan Brack Perilla · Camilo Céspedes Leguizamón

Este documento cubre los **entregables 2 y 4** del enunciado (documentación técnica e informe de resultados), que el enunciado permite fusionar.

Estado: BORRADOR del 10/09/2026. Contiene resultados reales y verificables de tres configuraciones medidas. Las secciones marcadas pendiente esperan las variantes A y C.

1. Resumen ejecutivo

Se construyó un servicio que responde a un estímulo de 32 bytes con una respuesta de 32 bytes, y se midió su latencia de ida y vuelta bajo una metodología única. El objetivo del enunciado era **una latencia inferior a 1 ms**.

El objetivo se alcanza trivialmente. La configuración más lenta que medimos —un servidor TCP escrito en Python— responde con una mediana de **13,2 μ s**, unas **75 veces por debajo** del umbral. Conseguir el número no era el reto.

El trabajo real fue otro. Medimos **tres configuraciones bajo condiciones idénticas** y obtuvimos tres resultados que no se ven mirando el promedio:

Hallazgo	Evidencia
El lenguaje de programación era irrelevante; la capa de comunicación lo era todo.	Reescribir el servicio de Python a C, con el mismo transporte, mejoró un 9,2 % . Cambiar el transporte, con el mismo lenguaje, mejoró 144,6x .
Dos de las tres configuraciones incumplen el objetivo, y solo se ve en la cola.	Por mediana las tres cumplen con holgura. Por máximo, dos superan 1 ms en 363 y 141 casos de 3 000 000.
El máximo no es una propiedad del sistema sino de cuánto se observe.	La misma configuración dio un máximo de 2,2 ms con 100 000 muestras y de 44 ms con 3 000 000. No cambió el sistema: cambió la ventana.
Eliminar el software de la ruta crítica no elimina la latencia impredecible.	La configuración sin una sola llamada al sistema operativo sigue teniendo máximos de 35 μ s, 420x su mediana. La imponen el planificador y el hardware.

Conclusión arquitectónica: el reto no se gana optimizando, se gana **decidiendo en qué capa está el coste** y aceptando explícitamente qué se sacrifica a cambio.

2. El problema y los drivers arquitectónicos

2.1 Requisitos, extraídos del enunciado

#	Driver	Origen textual	Tipo
D1	Latencia de ida y vuelta < 1 ms	«preferiblemente menor a un milisegundo»	Atributo de calidad
D2	El sistema escucha permanentemente	«debe escuchar permanentemente peticiones»	Restricción funcional
D3	Ante un estímulo, retorna una respuesta específica	«retornar una respuesta específica»	Requisito funcional
D4	La latencia debe medirse	«debe medirse el tiempo transcurrido»	Observabilidad
D5	Las herramientas deben justificarse	«justificación de herramientas, lenguajes y metodologías»	Proceso
D6	Comparación explícita contra 1 ms	«comparación explícita con el objetivo de 1 ms»	Reporte

2.2 Lo que el enunciado no pide — y por qué importa

El enunciado **no** exige concurrencia, durabilidad, disponibilidad, red física entre máquinas, ni seguridad. Esa ausencia define el alcance tanto como lo que sí pide, y es la justificación formal para **no** incorporar contenedores, frameworks, brokers de mensajes, balanceadores ni TLS: ninguno reduce la latencia y todos la aumentan.

2.3 Supuestos declarados

#	Supuesto	Riesgo si es falso
S1	Mensajes de tamaño fijo y pequeño (32 B)	Bajo: por debajo del MTU el orden de magnitud no cambia
S2	Un solo cliente, una petición en vuelo	Medio: cambia la interpretación; se declara en §5.2
S3	Cliente y servidor en el mismo host	Alto: es lo que más baja el número. Ver §8.2
S4	Sistema en reposo durante la medición	Medio: infla la cola; se registra la carga

S3 se declara en voz alta y no se esconde. Medir en loopback elimina la red, que en un sistema real es el componente dominante. Presentar 13 μ s como «latencia del sistema» sin decir que no hay red de por medio sería deshonesto.

2.4 Atributos de calidad — escenarios medibles

Un atributo sin medida de respuesta es un deseo, no un requisito.

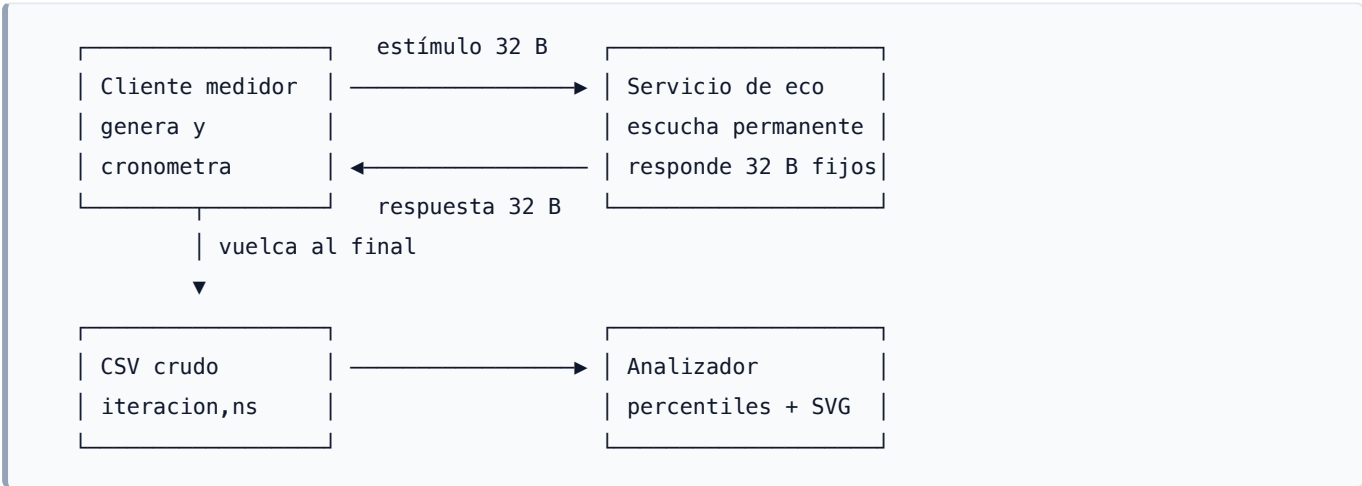
AC-1 · Latencia. Ante un estímulo de 32 B con el sistema en reposo y una petición en vuelo, **el p99.9 del RTT debe ser inferior a 1 ms**, medido en espacio de usuario del cliente.

AC-2 · Determinismo. Sobre 1 000 000 de mensajes consecutivos, **la relación p99.9/p50 debe ser ≤ 10 y ninguna muestra debe superar 1 ms.** (Este atributo no lo pide el enunciado: lo añadimos porque en sistemas de baja latencia reales el requisito nunca es «que sea rápido» sino «que la cola sea acotada».)

AC-3 · Comparabilidad. Todas las configuraciones producen el mismo formato de muestras crudas y sus métricas las calcula **un único script**.

3. Arquitectura del sistema

3.1 Contexto



El analizador está **fuera de la ruta crítica por diseño**: calcular percentiles dentro del bucle mediría el analizador, no el sistema.

3.2 Las configuraciones medidas

La diferencia entre ellas es **cuánta pila de sistema atraviesa cada mensaje**:

A aplicación → parseo HTTP → TCP → IP → loopback → IP → TCP → parseo HTTP → aplicación

B aplicación → TCP → IP → loopback → IP → TCP → aplicación

C aplicación → buffer del kernel → aplicación

D aplicación → memoria física compartida → aplicación

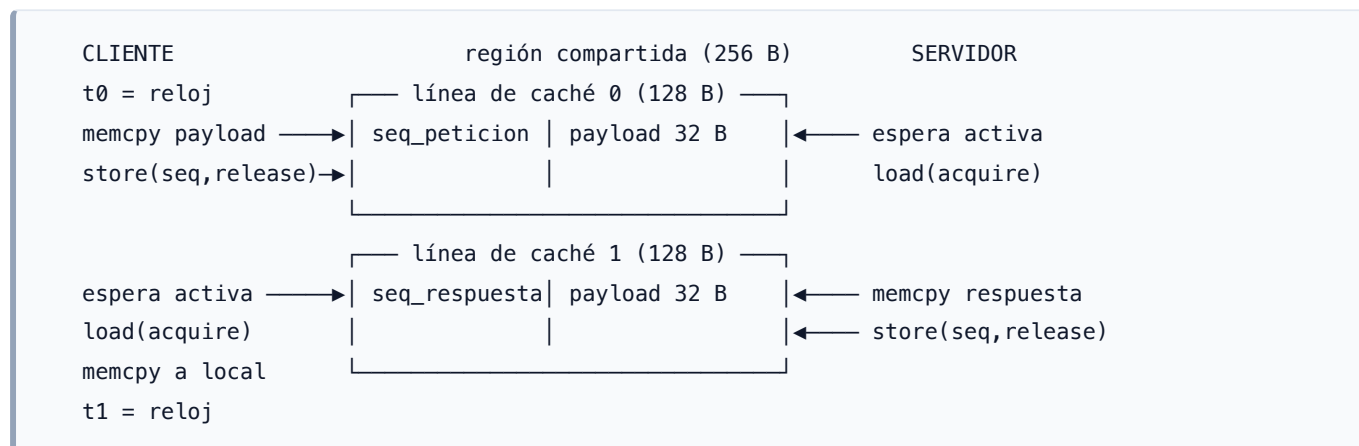
(sin llamadas al sistema en la ruta crítica)

ID	Transporte	Lenguaje	Estado
A	HTTP/1.1 (REST) sobre TCP loopback	por definir	pendiente pendiente
B	TCP crudo, TCP_NODELAY , conexión persistente	Python 3.13	✓ medida
Bc	(control) TCP crudo, idéntico a B	C11	✓ medida
C	Unix domain socket o UDP loopback	por definir	pendiente pendiente
D	Memoria compartida POSIX + espera activa	C11	✓ medida

Bc no es una cuarta arquitectura: es un experimento de control. Mantiene el transporte de B y cambia solo el lenguaje. Sin él, cualquier diferencia entre B y D mezcla dos variables y no se puede atribuir a ninguna. Es la pieza que permite la conclusión principal de §7.2.

3.3 La variante D en detalle

Dos procesos comparten 256 bytes de memoria física (`shm_open` + `mmap`). No hay sockets, no hay kernel en la ruta crítica, no hay planificador.



Tres decisiones que no son microoptimización sino corrección o un orden de magnitud:

1. **Cada sentido en su propia línea de caché (128 B en Apple Silicon).** Compartir línea provocaría *false sharing*: cada escritura de un proceso invalidaría la caché del otro.
2. **`_Atomic` con `acquire / release`, nunca `volatile`.** `arm64` tiene modelo de memoria débilmente ordenado: sin barreras, el otro núcleo puede ver la bandera antes que el dato. El mismo código con `volatile` funcionaría en x86 y **estaría roto aquí**.
3. **Espera activa que no cede el núcleo.** Ceder significa volver al planificador, que es exactamente el coste que esta configuración existe para eliminar. Precio: dos núcleos al 100 % permanentemente.

Lo que deliberadamente no tiene: ring buffer. Con una sola petición en vuelo añadiría índices y aritmética modular sin reducir latencia. Sería sobreingeniería.

4. Justificación de herramientas, lenguajes y metodologías

Requisito explícito del enunciado (entregable 2, D5).

Decisión	Alternativas descartadas	Por qué
Un estudio comparativo en vez de una implementación	Una sola implementación optimizada	El objetivo se cumple con 40 líneas. Cuatro personas en un servidor de eco se estorban, y el informe quedaría sin contenido arquitectónico. El temario del módulo es trade-offs; esto los ejercita, aquello no.
Python para B	C, Go, Java	Es el candidato más lento imaginable, y por eso el más informativo: si Python cumple el umbral, el umbral no es el reto. Además todos en el equipo lo tienen.
C11 para D	Java 21, Python	En Java el recolector de basura y el JIT introducen pausas en los percentiles que la configuración pretende demostrar. En Python el sobrecoste del intérprete ($\sim\mu s$) es mayor que la latencia a medir : mediría Python, no memoria compartida.
C11 también para el control Bc	—	Para aislar el lenguaje hay que cambiar solo el lenguaje.
Memoria compartida + espera activa para D	Semáforos o <i>futex</i> sobre memoria compartida	Reintroducen el planificador, que es lo que D quiere eliminar. D se solaparía con C y el estudio perdería un punto.
TCP_NODELAY en B y Bc	Dejar Nagle activo	Sin ello el kernel agrupa paquetes pequeños y aparecen picos de decenas de ms. Es el error clásico del ejercicio.
Conexión persistente	Conexión por petición	El handshake se paga una vez, en el calentamiento. Repartirlo por iteración falsearía.
SVG generado sin dependencias	matplotlib	El harness lo comparten cuatro personas: una dependencia obliga a los cuatro a instalarla y a que coincidan las versiones. Con la librería estándar cualquiera regenera las figuras idénticas.
Un único script de análisis	Que cada uno calcule sus percentiles	Sin esto los resultados no son comparables (AC-3), y la comparabilidad es la tesis del trabajo.

Lo que se decidió NO construir

Descartado	Por qué
Base de datos	Nada que persistir
Contenedores	Añaden red virtualizada: umentan la latencia
Framework web (B, C, D)	Capas de indirección en la ruta crítica
Broker de mensajes	Un salto de red y persistencia: tres órdenes de magnitud en contra
TLS / autenticación	Sin superficie de amenaza en el alcance; añade handshake y cifrado por mensaje
Reintentos, circuit breaker	Sin requisito de disponibilidad; añaden ramas en la ruta crítica

Justificar una ausencia es más difícil, y vale más, que justificar una presencia.

5. Cómo se mide la latencia

Requisito explícito del enunciado (entregable 2, D4). **Es la decisión arquitectónica central del trabajo:** el mismo sistema, sin cambiar una línea, reporta entre 5 μ s y 50 ms según dónde se coloquen las sondas.

5.1 La frontera de medición

Se adopta **F1 — RTT de aplicación medido en el cliente:**

```
t0 = reloj monótono, inmediatamente ANTES de escribir el estímulo
t1 = reloj monótono, inmediatamente DESPUÉS de tener la respuesta completa en buffer local
latencia = t1 - t0
```

Incluye serialización, escritura, transporte, procesamiento del servidor, transporte de vuelta, lectura completa y cualquier cambio de contexto ocurrido en medio. **Excluye** el establecimiento de la conexión (se paga una vez, en el calentamiento) y el arranque del proceso.

Fronteras consideradas y descartadas:

Frontera	Qué mide	Valor típico	Por qué se descartó
F1 ✓	RTT de aplicación completo	~13 μ s	<i>Es lo que percibe un consumidor real</i>
F2	Solo la llamada al sistema	~5 μ s	Esconde el coste de recibir; no tiene análogo en D, que no hace llamadas al sistema
F3	Incluye la conexión	~100 μ s	Con conexión persistente, repartir el handshake por iteración falsea
F4	Incluye el arranque del proceso	~50 ms	Mide el arranque del runtime, no el sistema
F5	Solo ida (cliente→servidor)	~6 μ s	Inválida: exige dos relojes sincronizados mejor que la magnitud medida. Entre procesos, el desfase es del mismo orden que la latencia

F1 es la más desfavorable de las defendibles. Elegir deliberadamente la medida que nos perjudica es lo que da credibilidad al cumplimiento. Podríamos haber reportado 5 μ s con F2, y habría sido igualmente «cierto».

Decisión completa y consecuencias: **ADR-001**.

5.2 Protocolo experimental

Parámetro	Valor	Por qué
Modo	Closed-loop, 1 petición en vuelo	Mide tiempo de servicio en vacío. Evita <i>coordinated omission</i>
Calentamiento	100 000 iteraciones, descartadas	Cachés, TLB, JIT, ramp-up de frecuencia de CPU
Medición	1 000 000 iteraciones × 3 rondas	3 000 000 de muestras por configuración
Payload	32 B fijos en ambos sentidos	Idéntico en todas o la comparación no vale
Almacenamiento	Array preasignado y pre-tocado	Asignar o provocar fallos de página en el bucle contaminaría
Volcado	Al final, nunca dentro del bucle	Escribir a disco mediría el disco

5.3 El instrumento: un problema que apareció al medir

La primera corrida de la variante D devolvió **ceros**. No era un fallo del código.

Granularidad real de cada reloj, medida en el equipo (200 000 pares de lecturas consecutivas):

Reloj	Granularidad	% de deltas nulos
<code>clock_gettime(CLOCK_MONOTONIC)</code>	1000 ns	97,6 %
<code>clock_gettime(CLOCK_MONOTONIC_RAW)</code>	41 ns	40,1 %
<code>clock_gettime_nsec_np(CLOCK_UPTIME_RAW)</code> ✓	41 ns	57,0 %
Contador de hardware (24 MHz)	41,67 ns	piso físico

«Resolución de nanosegundos» describe **las unidades del valor**, no **la granularidad con que avanza**. `CLOCK_MONOTONIC` devuelve nanosegundos y salta de microsegundo en microsegundo. Nuestra propia especificación de medición contenía ese error, y la variante D lo destapó.

Regla adoptada (ADR-003): la granularidad debe ser al menos **10x menor** que el p50 esperado. Si no lo es, la configuración añade un **contraste por lotes** como medida *secundaria* —nunca sustituta— de los percentiles.

Consecuencia reportada: las muestras de D están **cuantizadas en múltiplos de 41,67 ns**. No es ruido: es el tamaño del tic.

5.4 Entorno

CPU	Apple M4 — 10 núcleos (4 de rendimiento + 6 de eficiencia)
Arquitectura	arm64 , modelo de memoria débilmente ordenado
Sistema operativo	macOS 26.6.2 (Darwin 25.6.0), kernel de propósito general
Compilador	Apple clang 21.0.0, <code>-std=c11 -O2</code>
Runtime	Python 3.13.5
Fijación de hilos a núcleos	No disponible. <code>thread_policy_set(THREAD_AFFINITY_POLICY)</code> devuelve <code>KERN_NOT_SUPPORTED</code> (verificado con programa de prueba)

Las tres configuraciones se midieron en la misma máquina en reposo. Si se ejecutaran en equipos distintos, los números no serían comparables y el estudio perdería su tesis.

6. Resultados

Agregado de 3 rondas x 1 000 000 por fila. Todo en **nanosegundos**.

	B Python+TCP	Bc C+TCP (control)	D C+memoria compartida
n	3 000 000	3 000 000	3 000 000
mín	7 417	7 125	0
p50	13 209	12 000	83
p99	57 625	45 459	125
p99.9	184 083	107 458	167
p99.99	1 149 500	518 916	292
máx	44 020 084	29 423 042	34 833
media (no es la métrica principal)	15 579	13 556	70
muestras > 1 ms	363	141	0
Integridad	—	3 000 000 OK	3 000 000 OK

Figuras: harness/graficas/percentiles.svg y harness/graficas/histograma.svg . Se regeneran con python3 harness/graficas.py (sin dependencias externas).

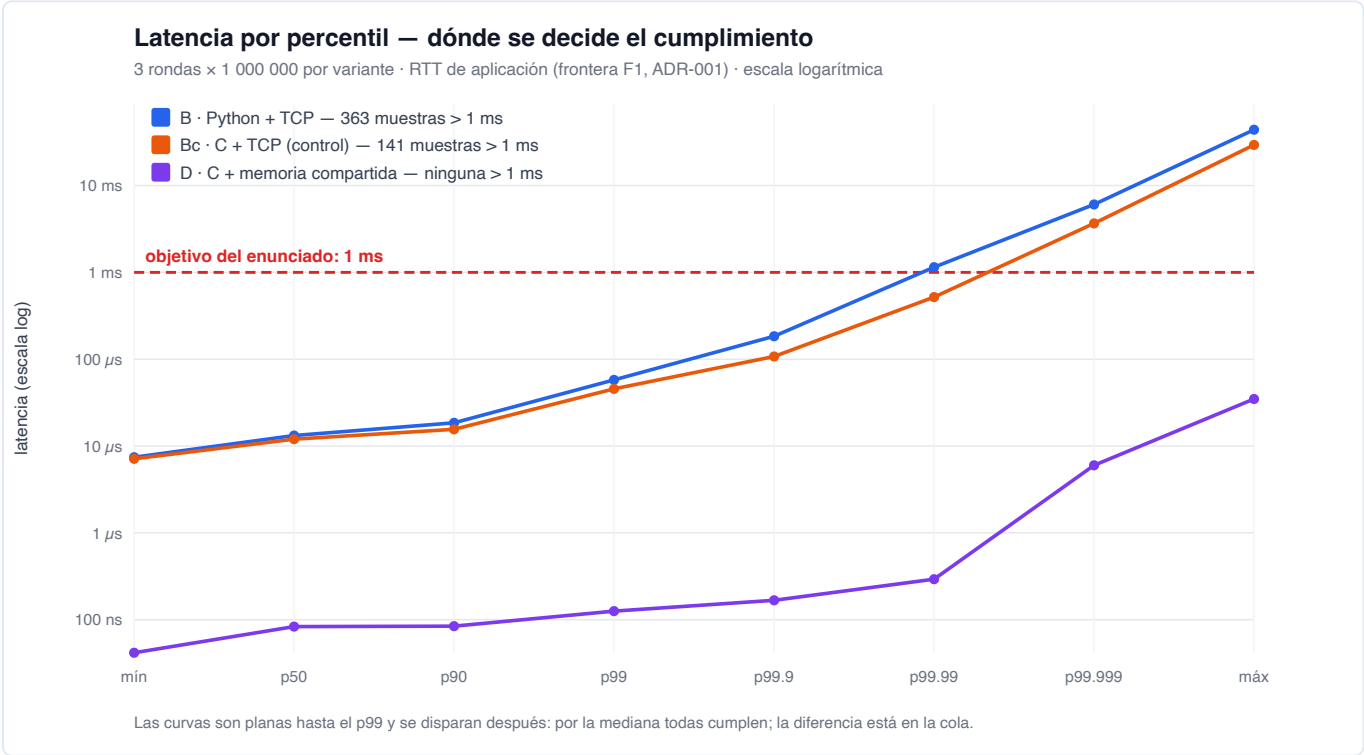
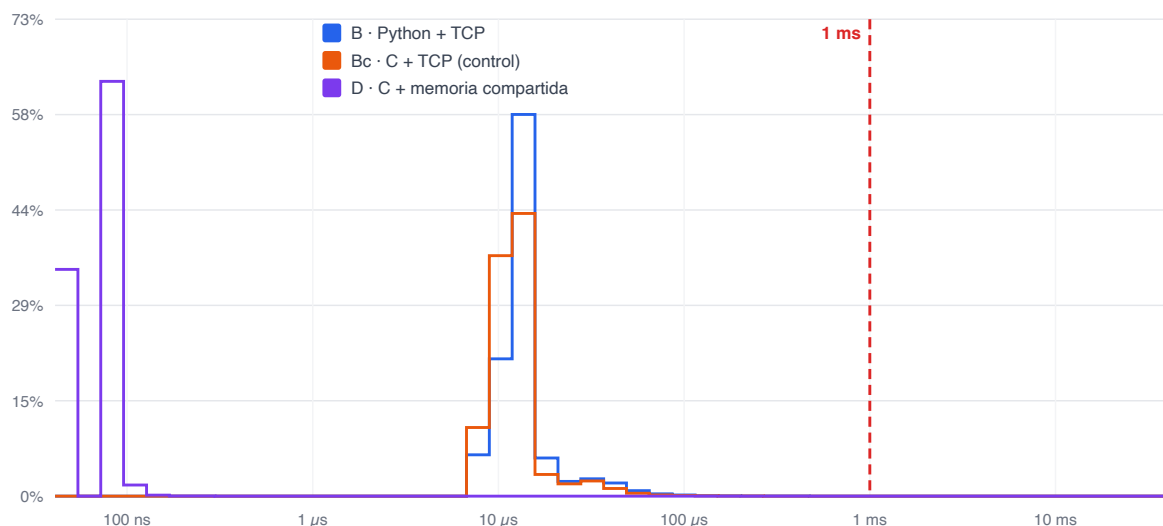


Figura 1 — Latencia por percentil. Por la mediana las tres cumplen; la diferencia está en la cola.

Distribución de la latencia — dónde cae cada arquitectura

porcentaje de muestras por cubo logarítmico · 3 000 000 de muestras por variante



La variante D no forma una campana sino dos picos: son los tics del reloj (41,67 ns). El instrumento es visible en el resultado.

Figura 2 — Distribución. La variante D no forma una campana sino dos picos: son los tics del reloj.

6.1 Contraste por lotes para la variante D

Cronometrando 1000 intercambios de una vez y dividiendo (200 rondas): mediana de **61, 61 y 68 ns** por intercambio, frente a los 83 ns del p50 por muestra.

La diferencia es explicable y no es contradicción:

- El p50 de 83 ns es el **tic superior** de la cuantización. La latencia real está entre 1 y 2 tics; los lotes la sitúan en ~61–68 ns. **El p50 sobreestima ~20 ns por cuantización.**
- La media por muestra (70,6 ns) menos la media por lotes (61 ns) da ~9 ns: **es el coste del par de lecturas del reloj.** A esta escala **el instrumento pesa el 15 %** de lo medido.

Se reportan las dos medidas, etiquetadas. Con una sola, o se pierde la cola (lotes) o se exagera la mediana (muestras). Ninguna es «la verdadera».

6.2 Distribución de la variante D — no es una campana

Valor	Tics	Muestras (ronda 1)	%
41,7 ns	1	329 034	32,90 %
83,3 ns	2	658 889	65,89 %
125,0 ns	3	10 054	1,01 %
≥ 166,7 ns	≥ 4	1 923	0,19 %

El 98,8 % de las muestras caen en dos cubos: son los tics del reloj. **El instrumento es visible en el resultado**, y por eso hay que declararlo.

6.3 pendiente Pendiente

Variantes A (HTTP/REST) y C (Unix socket / UDP). Con ellas el estudio cubrirá el rango completo de cuatro órdenes de magnitud y permitirá cuantificar por diferencia el coste de cada capa: protocolo de aplicación (A–B),

pila de red (B–C) y llamadas al sistema + planificador (C–D).

7. Análisis

7.1 Comparación explícita con el objetivo de 1 ms

Requisito explícito del enunciado (entregable 4, D6).

Por mediana, las tres configuraciones cumplen con holgura:

	p50	veces por debajo de 1 ms
B	13 209 ns	76x
Bc	12 000 ns	83x
D	83 ns	12 048x

Por la cola, dos de las tres incumplen:

	máx	muestras > 1 ms (de 3 M)	AC-1 (p99.9 < 1 ms)	AC-2 (ninguna > 1 ms)
B	44 020 084 ns	363	✓ cumple	✗ incumple
Bc	29 423 042 ns	141	✓ cumple	✗ incumple
D	34 833 ns	0	✓ cumple	✓ cumple

Esta es la comparación que el enunciado pide, y la respuesta depende de cómo se lea «lograr una latencia menor a 1 ms» :

- Si significa «la respuesta típica tarda menos de 1 ms» → **las tres cumplen**, y el reto se resuelve con 40 líneas de Python.
- Si significa «el sistema nunca tarda más de 1 ms» → **solo D cumple**, y la diferencia entre cumplir y no cumplir no tiene nada que ver con la velocidad media.

Un informe basado en promedios (15,6 μ s frente a 70 ns) habría dado las tres por buenas. **La métrica elegida, no el sistema, es lo que determina el veredicto.**

7.2 Dónde estaba realmente el coste

El control Bc permite descomponer el factor total:

Salto	Δ p50	Factor	Peso
Lenguaje (B → Bc): Python → C, mismo transporte	1 209 ns	1,1x	9,2 %
Transporte (Bc → D): TCP → memoria compartida, mismo lenguaje	11 917 ns	144,6x	90,8 %
Total (B → D)	13 126 ns	159,1x	100 %

Reescribir el servicio en C habría comprado un 9 %. Cambiar de capa de comunicación compró 144x. Y no habría arreglado el incumplimiento: Bc sigue teniendo 141 muestras sobre el umbral, frente a las 363 de B.

El intérprete de Python añade $\sim 1,2 \mu s$ por intercambio: es real y medible, pero está sepultado bajo los $\sim 12 \mu s$ que cuestan las dos llamadas al sistema, las copias del kernel, la pila TCP/IP y el despertar del planificador — coste que C paga igual.

Este es el resultado central del trabajo: se optimizó la capa equivocada. Ante un requisito de latencia, el reflejo habitual es cambiar de lenguaje. Los datos muestran que el 91 % del coste estaba en una decisión de arquitectura, no de implementación. Es exactamente la distinción que el Módulo 1 plantea entre decisiones estructurales y detalles de implementación.

Matiz obligatorio, para no convertir el hallazgo en consigna: el lenguaje pesa poco *en este servicio*, que no hace trabajo útil —recibe 32 bytes y devuelve 32 bytes—. Con lógica de negocio, serialización o acceso a datos, la proporción cambia. Lo demostrado es que **el transporte domina cuando el trabajo útil es despreciable**, no que el lenguaje nunca importe.

7.3 El máximo depende de cuánto se mire

Muestras observadas de B	Máximo
100 000 (corrida del 08/09)	2 202 000 ns
3 000 000 (rondas 1–3)	44 020 084 ns — 20x peor

No cambió el sistema: cambió la ventana de observación. Al medir más se capturan eventos más raros (planificación, interrupciones, presión de memoria, termorregulación).

Tres consecuencias:

1. **Un máximo sin su `n` al lado no significa nada.** Por eso `n` aparece en todas las tablas.
2. **Por eso los sistemas serios se especifican en percentiles.** El p99.9 de Bc se mantuvo entre 97 y 114 μs entre rondas, mientras su máximo saltaba de 13 a 29 ms. El percentil es estable; el máximo es una anécdota.
3. Cualquier afirmación del tipo «nunca supera X» es, en rigor, «no lo superó en las N muestras que observamos».

7.4 La cola persiste aunque se elimine el software

La variante D no hace **una sola llamada al sistema** en la ruta crítica, no copia por el kernel y no cede el núcleo. Aun así:

```
p50 =      83 ns
máx = 34 833 ns  →  420x la mediana
```

No lo causa el transporte. Lo causan el planificador del sistema operativo, las interrupciones y, muy probablemente, la migración del hilo a un núcleo de eficiencia — que en macOS/arm64 **no se puede impedir**, porque la afinidad de núcleo no existe (`KERN_NOT_SUPPORTED` , verificado). Lo único disponible es la clase de calidad de servicio `QOS_CLASS_USER_INTERACTIVE` , que es una sugerencia al planificador, no una garantía.

El sistema operativo y el hardware son restricciones arquitectónicas de primer orden, no detalles de despliegue. Es el frente «tecnología» de la taxonomía de restricciones del módulo, demostrado aquí con medidas propias en lugar de con una definición.

8. Conclusiones y trade-offs

8.1 Ninguna arquitectura gana

	A HTTP	B/Bc TCP	C IPC	D memoria compartida
Latencia p50	pendiente ~50–200 μ s	12–13 μ s	pendiente ~10–30 μ s	83 ns
Llamadas al sistema por RTT	≥ 2 + parseo	2	2	0
Funciona entre máquinas	✓	✓	UDP sí, Unix no	✗
Interoperabilidad	✓ alta	media	baja	✗ nula
CPU en reposo	baja	baja	baja	✗ 2 núcleos al 100 %
Complejidad	baja	baja	baja	alta
Depurable con herramientas comunes	✓	✓	parcial	✗

La pregunta correcta no es cuál es mejor, sino qué atributo prioriza el negocio:

- Prioriza **interoperabilidad y velocidad de desarrollo** → **A**. Los 200 μ s son irrelevantes para el 99 % de los sistemas de negocio.
- Prioriza **latencia con red de por medio** → **B o C**.
- Prioriza **determinismo extremo en un solo host** y puede pagar un núcleo dedicado, código no portable y nula interoperabilidad → **D**.

La trampa del ejercicio: el enunciado premia el número bajo, y el número bajo lo gana D. Pero D es la peor arquitectura posible para casi cualquier sistema real. Concluir «usen memoria compartida» sería haber aprendido a optimizar, no a arquitecturar. **El umbral de 1 ms ya lo cumple la opción más simple y portable**; pasar de A a D compra tres órdenes de magnitud que casi nadie necesita, al precio de todos los demás atributos.

8.2 Atributos sacrificados por la variante D

Atributo	Se sacrifica	A cambio de
Portabilidad	Depende del SO y de la arquitectura de CPU	2 órdenes de magnitud de latencia
Escalabilidad	Un solo cliente; dos núcleos al 100 %	Eliminar el despertar del planificador
Eficiencia de recursos	Dos núcleos quemados sin trabajo útil	Determinismo en la cola
Interoperabilidad	Solo entre procesos del mismo host	Evitar la pila de red completa
Mantenibilidad	Código de bajo nivel, errores de visibilidad silenciosos	Control sobre cada copia y cada barrera
Seguridad	Sin autenticación, cifrado ni validación	Sin superficie de amenaza en el alcance declarado

Esta tabla responde a «¿por qué no se usa esto en producción para todo?».

9. Limitaciones

1. **Sin red física.** Todo se midió en loopback (S3). En un sistema real la red domina la latencia, y ninguna de estas cifras la incluye.
2. **Sin carga concurrente.** Closed-loop con una petición en vuelo (S2). Estas cifras son latencia en vacío, no latencia bajo carga.
3. **Sin afinidad de núcleo.** No disponible en la plataforma. En Linux los resultados de D probablemente tendrían mejor cola; **no lo afirmamos porque no lo medimos.**
4. **Precisión acotada por el hardware.** Las muestras de D están cuantizadas en 41,67 ns: es el piso físico del contador de la máquina, no una limitación del código.
5. **Una sola máquina, un solo modelo de CPU.** Los factores relativos deberían mantenerse en otros equipos; los valores absolutos no.
6. **Dos de las cuatro variantes faltan.** El estudio está incompleto hasta tener A y C.

Trabajo futuro

- Completar A y C.
- Repetir la variante D en Linux con afinidad real de núcleo y comparar colas.
- Medir entre dos máquinas físicas para cuantificar cuánto aporta la red.
- Medir bajo carga concurrente, que es el escenario realista.

10. Anexos

10.1 Decisiones arquitectónicas registradas

ADR	Decisión
ADR-001	Frontera de medición F1 — RTT de aplicación en el cliente
ADR-002	Variante D: memoria compartida + espera activa, C11, QoS en lugar de afinidad
ADR-003	Resolución del reloj: enmienda a la especificación de medición

10.2 Reproducibilidad

```
cd harness
./run.sh B 1 --warmup 100000 --iters 1000000    # TCP en Python
./run.sh Bc 1 --port 9111 --warmup 100000 --iters 1000000    # control: TCP en C
./run.sh D 1 --warmup 100000 --iters 1000000    # memoria compartida en C
./analyze.py --md resultados/*.csv              # tabla comparativa
./graficas.py                                   # figuras SVG del informe
```

`run.sh` compila si hay `Makefile`, registra CPU, sistema operativo, núcleos, compilador y runtime en el log, y ejecuta el mismo analizador para todas las configuraciones.

10.3 Evidencia

Entregable del enunciado	Ubicación
1 · Código fuente	harness/ completo
2 · Documentación técnica	este documento, §§ 2–5
3 · Logs de ejecución	harness/resultados/ejecucion-*.log
4 · Informe de resultados vs 1 ms	este documento, §§ 6–8
5 · Video ≤ 5 min o demo en vivo	<i>pendiente</i> pendiente

Muestras crudas: harness/resultados/resultados- $\{B,Bc,D\}$ - $\{1,2,3\}$.csv , 9 000 000 de filas.